

MarketsGrid — Feature Overview

What MarketsGrid is

It's a "power user" data grid built on top of AG-Grid Enterprise (v33+), aimed at financial-markets-style use: lots of rows, live updating data, every user wanting to bend the grid to their own taste, and those tastes needing to survive reloads, layout switches, and even being moved to a separate window.

Think of it as **Excel-meets-trading-blotter**, where every column, color, filter, and formula a user sets up gets saved into a named "layout" they can switch between.

The grid surface itself

- A high-performance data grid with everything AG-Grid Enterprise gives you: sorting, filtering, grouping, pivoting, pagination, side bar, status bar, frozen rows/columns, virtual scrolling, copy/paste, Excel export, and so on.
- Optimised for **streaming data** (prices ticking ~30 times a second): updates are coalesced into 100ms batches, filter inputs don't lose focus mid-keystroke when data ticks, and conditional-style repaints only touch the cells that actually changed.
- Cells, rows, the whole grid, plus its chrome (toolbars, popovers, dialogs) all respect a **dark/light theme** that flips by changing one attribute on the page.

The toolbar across the top

Everything in the strip above the grid:

- **Editable title** ("caption") — click the pencil icon to rename what this grid is called.
- **Filter pills bar** (optional) — saved filters appear as clickable chips.
- **Formatting toolbar toggle** (optional) — opens a second strip for styling cells.
- **Layout picker** — switch between saved looks of the grid.
- **Save button** — saves everything you've changed. Glows when there are unsaved changes, shows a checkmark briefly when the save lands.
- **Settings gear** — opens the settings drawer.
- **Admin buttons** — optional extra icon buttons the host app can plug in (e.g. "Open Config Browser", "Audit Log").
- **Info ⓘ popover** — shows technical identifiers for support/debugging.

Layouts (the headline feature)

A layout is **one saved combination of every customization** described below — columns, filters, styling rules, calculated columns, toolbar layout, sort order, the lot.

Users can:

- Create, rename, clone, and delete layouts.
- Switch layouts. If they have unsaved edits a dialog asks "Save & switch / Discard changes / Cancel".
- Export a layout to a JSON file and import one back.
- Be warned by the browser if they try to close the tab with unsaved changes.
- (Inside OpenFin) Have each open window remember its own active layout, so two windows showing the same grid can show different layouts.

Saving is **explicit** — there is no auto-save scribbling silently in the background, because that confuses people about what their layout actually contains.

The filter-pills bar

A row of chips where each chip is a saved filter:

- Save the current filter as a new chip (auto-named from the filter content, with a numbered fallback).
- Click a chip to toggle it on/off; multiple chips combine.
- Each chip shows a **row-count badge** so you can see "this filter matches 47 rows" before applying it.
- Rename a chip inline; delete it; edit the underlying filter JSON in a popover.
- Collapse the whole bar into a compact "3 filters · 1 active" summary chip.
- Scrolls horizontally with arrow buttons when there's too many to fit.

The formatting toolbar

A second strip (toggled from the main toolbar) that styles whatever cell/column/header is currently selected. Quick-access pills for:

- **Context** — switch between styling cells vs styling headers, and pick which column you're targeting.
- **Type** — bold, italic, underline, alignment, font size.
- **Paint** — text color, fill color, borders.
- **Format** — number formatting: currency (multiple currencies), percent, basis points, decimals up/down, comma separators, bond ticks (32nds, 64ths, 128ths, 256ths), or a custom Excel-style format string.
- **Library** — save the current set of style choices as a reusable **template** that you can apply to other columns in one click.
- **Clear** — reset some or all formatting.

The whole toolbar can be "**popped out**" into its own always-on-top OS window so you can keep it visible while clicking around the grid.

The Settings drawer

A side panel (also poppable into a separate window) with one tab per topic:

1. **Grid Options** — about 40 of AG-Grid's most useful global settings: row height, header height, pagination, multi-sort mode (replace/shift/ctrl/always), row selection mode, checkbox column, cell selection, drag-and-drop reorder, cell flash duration, side bar setup, status bar panels (total/filtered/selected counts, aggregations), advanced filter, tooltips, undo/redo, animation toggles, performance knobs, and column defaults.
2. **Column Templates** — define reusable presets you can apply to many columns at once (e.g. "Currency Column" = right-aligned, 2 decimals, dollar sign, light grey background).
3. **Column Settings** — for every column, override its header name, width, pinned side, hidden state, header style, cell style, alignment, value formatter, filter type (text/number/set/date/multi), cell editor (text/number/dropdown/date/large text/popup), and row-grouping behavior.
4. **Calculated Columns** — define virtual columns whose value is computed from a formula referencing other columns (`[price] * [quantity]` , `SUM([volume])` , etc.). They appear like real columns and can be styled and filtered.
5. **Column Groups** — nest columns under group headers (e.g. group "Bid Price" + "Ask Price" under a "Quotes" header). Groups can themselves be styled and can remember whether they're expanded or collapsed.
6. **Conditional Styling** — IF/THEN rules: "if price went up, flash the cell green for 800ms"; "if P&L is negative, paint the row red with a warning icon"; "if column X is above its average, show an up-arrow in the header." Supports cell-scope and row-scope rules, expression-driven matching, flash animations, header indicators, and time-limited (expiring) activations.
7. **Saved Filters** — the same chips as the filter bar, manageable here too.
8. **Toolbar Visibility & Grid State** — under the hood; keeps your toolbar layout and the grid's native column order / widths / sort / filter / pagination / sidebar state in your layout so a save-then-reload restores the exact view.

Storage & integration

- **Pluggable storage** — out of the box uses in-memory (which warns you in dev that things will vanish on reload). Apps can wire IndexedDB or a corporate config service.
- **Scoped storage** — layouts are keyed by (app id, user id, instance id), so two users on the same machine never see each other's layouts, and two grids in the same app don't collide.
- **"Grid-level" data slot** — for state that should survive layout switches (e.g. which data provider you're connected to).

- **Imperative handle** — host apps get a `ref` exposing the AG-Grid API, our module platform, the layout manager, and a `saveAll()` function, so they can trigger save from outside (like an OpenFin "Save Workspace" command).
- **App-data binding** — cell-editor dropdowns can pull their options from the host app's data registry via simple `{{providerName.key}}` references.

Quality-of-life details

A few things that won't show in feature lists but make the experience feel polished:

- Save flashes a checkmark for 600ms so you know it worked.
- Settings/Formatting popouts that are buried behind other windows get raised to front instead of opening a duplicate.
- Layout clones auto-suffix with " (copy)", " (copy 2)", " (copy 3)" to avoid name collisions.
- Malformed filters from old layout snapshots are repaired or quietly dropped instead of crashing the grid.
- Strict-mode safe, OpenFin-aware, framework-agnostic at its host boundary.

Addendum: How MarketsGrid plugs into ConfigService, Data Providers, and OpenFin

These three systems sit underneath MarketsGrid and turn it from "a clever grid" into "the standard grid for the whole trading desktop." Each is optional — MarketsGrid runs standalone in a browser with none of them — but together they're how the production app actually behaves.

1. Config Manager (the durable store)

The platform has a shared service called **ConfigService** (package: `@starui/config-service`) that holds every kind of configuration the app cares about — registry of launchable components, dock buttons, data-provider definitions, saved workspaces, user profiles, roles, and MarketsGrid layouts. Locally it lives in **IndexedDB via Dexie**; in production it's backed by a **REST API**, with the same Dexie copy used as a read cache.

MarketsGrid plugs into it via a small adapter factory called `createConfigServiceStorage(configManager)`. Passing that factory to `<MarketsGrid storage={...}>` switches the grid from in-memory storage to ConfigService.

What that wiring actually does:

- **One row per instance, all layouts bundled.** A grid's entire layout set lives in a single `AppConfigRow` keyed by `(appId, userId, instanceId)`. The row's `payload` is `{ version, profiles: [...], gridLevelData }`. This is intentional — admins opening the **Config Browser** see one row per blotter ("alice's positions blotter, all her saved views in there"), not N rows per view.
- **Identity scopes everything.** `appId` (which app), `userId` (which signed-in user), `instanceId` (which window/view of that app) form the primary key. Two users on the same machine see independent layouts. Two windows of the same blotter component share layouts (so the user's customizations follow them around). A user switching identity (impersonation, login) rebuilds the adapter cleanly without re-creating the factory.
- **Optimistic concurrency.** Every save reads the row's `version`, writes back `version + 1`, and throws `ProfileSetVersionConflictError` if someone else (another tab, another machine) bumped the version in between. Stops two windows silently clobbering each other.
- **Identity-bound rows.** When the launch flow knows what kind of component this view is (e.g. "Positions Blotter v3" from the **Component Registry**), the saved row carries that registered `componentType` / `componentSubType` / `isTemplate` / `singleton` — exactly matching the Registry Editor entry, so admin tools and visibility rules work on the actual component type, not a generic "markets-grid-profile-set" placeholder.
- **Template-to-instance pre-clone.** When the launcher opens a new view, it copies the registered component's template layout row onto the freshly minted `instanceId` **before** the view's `MarketsGrid` mounts. The grid hits a populated row on first read — no flicker, no seed race.
- **gridLevelData rides in the same row.** Provider selection (see below), persisted caption, and anything else the host wants to keep across layout switches lives at the top of the bundle, never inside an individual layout.
- **Migration helper.** `migrateProfilesToConfigService({ source, target, ... })` is a one-shot admin action that copies layouts out of an old Dexie/memory adapter and into the bundled `ConfigService` row, with skip-if-exists or overwrite strategies.

2. Data Providers (the live data plane)

The data side is `@starui/data-services`. Most production grids aren't given a static `rowData` array — they're wrapped in `<MarketsGridContainer>` (package: `@starui/widgets-react/v2`), which is a layer that knows how to talk to data providers.

Architecture in one paragraph: a **SharedWorker** (`SharedWorkerDataServicesHub`) runs once per origin and owns every active subscription. Browser windows / OpenFin views are thin **clients** of that worker. So if you open three windows of the same blotter, only one upstream connection runs; all three windows share the worker's cache and updates fan out to each of them.

How `MarketsGrid` sees this:

- **Two provider slots per grid.** A "Live" provider (typically STOMP/WebSocket — streaming) and a "Historical" provider (typically REST with an as-of-date). The user can have both configured at once and **toggle which mode is active** from a small `ProviderToolbar` mounted in `MarketsGrid`'s `headerExtras` slot.
- **Toolbar is dev/support-flavoured.** It's shown by default but hidden behind **Alt+Shift+P** (the original `Ctrl+Shift+P` clashed with browsers' "incognito window" shortcut). End-user grids that hardcode their provider just don't show it.
- **Selection persists as `gridLevelData`.** The picker writes `{ liveProviderId, historicalProviderId, mode }` to `MarketsGrid`'s grid-level slot, so switching layouts doesn't lose your provider, but switching users does (it's stored per-`(appId, userId, instanceId)`).
- **Two-phase data flow.** On every (re)subscribe: the worker delivers a **snapshot** in chunks (which the container accumulates into a single `setGridOption('rowData', ...)` call when the provider transitions `loading → ready`), then **live updates** flow as add/update transactions on AG-Grid's async transaction queue (coalesced into 100ms batches). Add-vs-update is decided by composing the row id from `keyColumn` exactly the way AG-Grid's `getRowId` would, so the worker cache and the grid stay byte-for-byte consistent.
- **Refresh = re-subscribe with a hint.** Hitting the Refresh button (in the toolbar or as a built-in admin action) passes `{__refresh: <timestamp>}` (live) or `{asOfDate}` (historical) as the `extra` payload; the worker calls `provider.restart(extra)`, clears the cache, and re-snapshots. **Every** connected window sees the loading overlay during a peer-triggered refresh — it's a single source of truth.
- **AppData — named key/value store for cross-component values.** A separate runtime piece called `AppDataStore` holds named entries like `positions.asOfDate`, `users.currentRegion`, `mode.theme`. Two ways it plugs into `MarketsGrid`:
 - **Provider config templates.** A historical provider's config can reference `{{positions.asOfDate}}` in its URL or body; the container's date picker writes the user-picked date to that `AppData` entry, `useResolvedCfg` re-resolves the template, and the worker triggers a re-subscribe automatically.
 - **Cell-editor `valuesSource`.** A dropdown cell editor on any column can specify `valuesSource: '{{currencies.list}}'`; at edit time `MarketsGrid`'s platform looks up that `AppData` entry and uses its keys as the dropdown options. Lets admins change a dropdown's contents without touching the grid.
- **Provider configs are themselves `ConfigService` rows** (componentType `'data-provider'`), edited through the Config Browser or its dedicated editor. So defining a new data feed is the same conceptual operation as defining a layout — it's just another configuration owned by an app/user.

3. OpenFin (the desktop shell)

The production app runs inside **OpenFin** — a Chromium-based desktop runtime that gives the app multiple OS-level windows, a dock, an inter-application bus, and the ability to save and restore the user's entire workspace. MarketsGrid is **OpenFin-aware but not OpenFin-required**: every integration falls through cleanly when `fin` is undefined.

The integrations:

- **Per-view active-layout pointer.** Each OpenFin **view** stamps its currently-active layout id onto its own `customData` (via `fin.me.updateOptions(...)`). This means a user can **duplicate a view** of the same MarketsGrid instance and have the two copies show **different layouts** — useful for side-by-side comparison ("show me yesterday's columns vs today's"). Outside OpenFin this hook returns `null` and the grid falls back to a single `localStorage` pointer.
- **Layouts round-trip through workspace snapshots automatically.** When the user saves a workspace, OpenFin's `Platform.getSnapshot()` captures each view's `customData` — including the active-layout id — into the workspace JSON. Reloading the workspace months later reopens each view bound to the layout it had when saved.
- **Saved workspaces ARE ConfigService rows.** The platform module (`openfin-platform`) overrides OpenFin's built-in workspace CRUD via `WorkspacePlatformOverrideCallback`, redirecting reads/writes to ConfigService rows of `componentType: 'workspace'`. So workspaces share the same backing store as everything else — same admin tools, same export/import, same role/visibility model.
- **Awaited save fan-out.** When the user clicks "Save Workspace", a channel called `marketsui-workspace-save-channel` dispatches a `workspace-saving` message to every connected view and **awaits** their responses. Each MarketsGrid-hosting view uses its imperative-handle `saveAll()` to flush unsaved layout edits **before** OpenFin captures the snapshot. No more "I saved my workspace but my column changes were gone."
- **Settings drawer + Formatting toolbar pop out as real OS windows.** Both use a `<Poppable>` helper that, inside OpenFin, opens a frameless OpenFin window with `alwaysOnTop: true`. A custom draggable title bar replaces the native chrome (keeps the look uniform), and "buried popout" detection raises the existing window to front on re-click instead of opening a duplicate. In a plain browser, both fall back to ordinary popup windows.
- **Editable caption surfaces only when tabs are hidden.** When the OpenFin tab strip is visible, the tab IS the view's title, so MarketsGrid's caption stays hidden. Hide the tab strip and the caption appears at the left of the toolbar with a pencil icon — same affordance, different surface.

- **Dock buttons + component registry.** The dock is configured through `openfin-platform` (`updateDockButtons` , IAB topics like `IAB_REGISTRY_CONFIG_UPDATE` and `IAB_THEME_CHANGED`). Clicking a dock button typically calls `launchRegisteredComponent(componentId)` which:
 1. Mints a fresh `instanceId` .
 2. Pre-clones the registry's template layout row onto that id (so `MarketsGrid` hits a populated row at mount).
 3. Opens an OpenFin view pointed at the component's URL, with `customData` carrying `{ instanceId, appId, userId, registeredIdentity }` .
- **Identity flows top-down.** The view picks up `appId / userId / instanceId / registeredIdentity` from OpenFin's `customData` on launch (helper: `useHostedIdentity`), passes them to `MarketsGrid` as props, and they propagate into `ConfigService` writes, `AppData` scoping, and the data-provider worker's cache keys. Same user in two OpenFin windows of the same component = one cached subscription, one shared layout set.
- **Workspace GC.** Because each saved workspace records the `instanceIds` it references, a background job can drop orphan layout rows that no workspace points at any more — keeps the `ConfigService` database from growing unbounded as users create and discard windows.

How the three combine on a real launch

A user double-clicks the "Positions Blotter" dock button. In order:

1. **OpenFin** launches a new view from the registry entry, minting `instanceId = "positions-7f3a"` and stamping `customData = { appId, userId, instanceId, registeredIdentity }` .
2. The **registry launcher** copies the component's template layout row onto `(appId, userId, "positions-7f3a")` in **ConfigService**.
3. The view boots. `<HostedMarketsGrid>` reads identity from `customData` , builds a `createConfigServiceStorage(configManager)` factory, and renders `<MarketsGridContainer storage={...} appId={...} userId={...} instanceId="positions-7f3a">` .
4. **MarketsGridContainer** resolves the storage factory, reads `gridLevelData` for the persisted provider selection, mounts `<MarketsGrid>` with the active provider's column defs.
5. **MarketsGrid** reads the bundled row, picks the per-view active layout pointer off OpenFin's `customData` (falling back to `localStorage`, then `Default`), and applies the layout's saved column / filter / formatting / calculated-column state.
6. The **data-services client** asks the **SharedWorker** to subscribe. If another window already has this provider subscribed, the snapshot comes from the worker's cache (instant). Otherwise the worker connects upstream, streams the snapshot back in chunks, then live updates begin.
7. The user customizes some columns, hits Save. `MarketsGrid` captures AG-Grid's native state, bumps the bundle's `version` , writes through `configManager.saveConfig(row)` . `Dexie` persists locally; in REST mode, the manager also POSTs to the backend (with `If-Match: <version>` once that backend ships).

8. Hours later the user clicks "Save Workspace" on the OpenFin dock. The platform fans out `workspace-saving` ; this view's handler awaits `marketsGrid.saveAll()` , then OpenFin captures the snapshot. The snapshot — including `customData.activeProfileId` on every view — lands as a `ConfigService` `componentType: 'workspace'` row.
9. Next morning, restore the workspace. Every view comes back bound to the same `instanceId` , the same active layout, and (because identity tuples match) the same data-provider subscription is reused from the worker's cache if any other window already opened it.

That's the full integration loop. `MarketsGrid` is one node in it, but it's the one that holds the user's actual customizations — which is why so much of the surrounding plumbing exists to keep those customizations correctly scoped, durable, and reproducible across windows, machines, and reloads.

4. Angular parity — in flight

An **Angular version of `MarketsGrid` is being built alongside the React one**, with deliberate, enforced feature parity as a non-negotiable. The Angular grid is not "a separate product that happens to look similar" — it's the same product, on the same underlying infrastructure, with the same configuration storage, data plane, and OpenFin integration. A user switching between a React-based blotter and an Angular-based blotter in the same desktop workspace shouldn't be able to tell which framework rendered which window, and an admin shouldn't have to author anything twice.

What's already landed in the Angular workspace (`packages/angular/`):

- **`config-service-angular`** — the Angular client for `ConfigService` (`ConfigServiceClient` + DI tokens), so Angular components scope their configuration writes by `(appId, userId, instanceId)` exactly the way the React adapter does. Same backing rows, same `Config Browser` surface — admin tools work on either side.
- **`data-services-angular`** — Angular bindings around the same `SharedWorker` data-services runtime: `DataServicesService` , `injectAppData()` , `injectProviderStream()` , `injectResolvedCfg()` , `injectDataProviderConfig()` . Mirror of the React hooks, same provider/AppData semantics, **same worker** — so an Angular view and a React view of the same provider share one cached subscription.
- **`host-wrapper-angular`** — the Angular equivalent of the React hosted-identity helper: reads `appId / userId / instanceId / registeredIdentity` from OpenFin's `customData` and exposes them through DI tokens. Identity flows top-down the same way; same rules, same scopes.
- **`widgets-angular`** — the Angular widget set, including the **Data Provider Editor** (live + historical provider forms, STOMP / REST / mock subtypes, field inference) and the **Dock Configurator**. Both write to the same `ConfigService` rows their React counterparts read.
- **`config-browser-angular`** — the Angular `Config Browser` tool, showing the same `AppConfigRow` tables the React version operates on.

What's coming — the Angular **MarketsGrid widget itself** (the equivalent of `@starui/markets-grid` and `@starui/grid-react`), parity item by parity item: layouts, the same nine modules (Grid Options, Column Templates, Column Settings, Calculated Columns, Column Groups, Conditional Styling, Saved Filters, Toolbar Visibility, Grid State), the same filter-pills bar, the same Excel-style formatter, the same popoutable settings drawer and formatting toolbar, the same OpenFin-aware per-view active-layout pointer.

The infrastructure-first ordering is deliberate. Shipping the Angular config client, data-services client, and host wrapper **before** the grid widget itself means that by the time the Angular MarketsGrid lands, it inherits everything described in this addendum for free — ConfigService persistence, optimistic concurrency, SharedWorker-backed data subscriptions, AppData binding, workspace-snapshot integration, identity-scoped rows, awaited workspace-save fan-out. The grid widget becomes the last brick in a wall that's already standing, rather than a green-field project that has to rebuild plumbing from scratch.

The non-negotiable parts of the rule (from the platform's working conventions):

- **No drift in stored shape.** Angular and React must read and write the same `AppConfigRow` payload byte-for-byte. A layout saved by the React grid must load in the Angular grid (and vice versa) with no migration step.
- **No drift in data semantics.** Same `keyColumn` composition, same `getRowId` rules, same `{{providerName.key}}` template resolution, same as-of-date round-trip into AppData.
- **No drift in identity.** Same `(appId, userId, instanceId)` tuple, sourced from the same place (OpenFin `customData` in production), so the same user in an Angular window and a React window of the same registered component sees the same layout set.
- **No drift in user-facing features.** Every module the React grid offers — calculated columns, conditional styling rules, column groups, the formatter's currency / percent / tick / Excel-format outputs, the popoutable settings drawer — has an Angular counterpart on the roadmap. The Angular bucket uses **PrimeNG** (themed via `@starui/tokens-primeng`) where React uses `shadcn/ui`; the visual chrome differs, but the capability surface doesn't.

In short: the Angular MarketsGrid is the immediate priority on the Angular side, the infrastructure under it has already been built out so the grid itself can land without re-inventing storage / data / identity / OpenFin plumbing, and feature parity with the React grid is treated as a hard constraint, not an aspiration.